

Formal Validation Closes the Gap Between AI Code Generation and Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

Significance. Large language models can generate numerical simulation code from plain-language descriptions, but they produce silently wrong results on the majority of non-textbook scientific problems because they lack formal mathematical checks. We introduce a validation layer—implemented as an agent that executes well-posedness proofs, stability checks, and error-budget computations—and show that inserting it between AI planning and AI execution reduces the silent-failure rate from 58% to below 6% across three deeply validated scientific domains (medical imaging, seismic inversion, combustion chemistry) with real measured data. The validated pipeline achieves 95–99% of domain-expert solution quality at 1/500th the setup time, and a prospective benchmark of 72 externally contributed, previously unseen tasks confirms generalization beyond the development set.

Abstract. AI-generated code can now produce numerical solvers from natural-language problem descriptions, yet it does so without error certificates—leaving scientists unable to distinguish correct from incorrect output. Here we show that a formally grounded validation step, inserted between problem specification and numerical execution, is both necessary and sufficient to close this gap. We define a *simulability class* $\mathfrak{S}$ of problems admitting bounded-error numerical solutions (Definition 1) and prove that, for problems in $\mathfrak{S}$ whose discretization decomposes over a 12-operation primitive basis, a three-agent pipeline (Plan, Validate, Compute) produces solutions satisfying $\|\hat{x} - x^*\| \leq \varepsilon$ for any prescribed tolerance $\varepsilon > 0$ (Theorem 2). The theorem rests on classical convergence theory (Lax–Richtmyer); the contribution is an automated pipeline that *identifies and constructs* the convergent discretization from natural language, with each agent addressing a distinct failure mode. We validate deeply in three domains with real measured data: clinical CT reconstruction (200 cases, 99% of expert quality), seismic full-waveform inversion on the Marmousi-2 benchmark (95% of expert quality), and GRI-Mech 3.0 combustion ignition delay against shock-tube measurements (6.3% error). A controlled comparison against GPT-4—given both raw descriptions and structured specifications—shows that AI code generation without formal validation silently fails on 58% of frontier problems; our Validation Agent reduces this to below 6%. A prospective benchmark of 72 tasks contributed by 12 external scientists, none involved in development, confirms that the pipeline generalizes to unseen domains and problem types. Code, specifications, benchmark tasks, and execution logs are publicly available.

1 Introduction

The problem. Large language models can now generate numerical simulation code from natural-language descriptions of scientific problems [7, 8]. A researcher describes a differential equation, and the model produces a working solver—often in seconds. This capability could democratize computational science by removing the programming barrier that prevents most domain scientists from running simulations—much as AlphaFold [6] democratized protein structure prediction. But there is a critical gap: the generated code comes with no guarantee that its output is correct. In a scientific culture already facing a reproducibility crisis [9], silent computational errors compound the problem. When the code produces a wrong answer—a laminar solution for a turbulent flow [18], a missing electron-correlation term in a quantum calculation, a numerically

unstable time integrator for a stiff chemical system—the scientist has no way to detect the error.

The claim. We show that formal mathematical validation, implemented as an autonomous agent that executes well-posedness proofs and error-budget computations, is necessary and sufficient to close this gap for the class of problems $\mathfrak{S}$ defined below. “Necessary” is demonstrated by ablation: removing the Validation Agent causes the pipeline to produce silently wrong results on 42% of tested problems (Table 2, “No $\mathcal{A}_V$ ” column). “Sufficient” is demonstrated by a controlled comparison: inserting the Validation Agent reduces the silent-failure rate of AI-generated solvers from 58% (GPT-4 alone) to below 6% (Table 2), while providing formal error certificates that raw code generation never produces.

What this paper does not claim. We do not claim universality. The theorem holds for problems whose discretization decomposes over a specific 12-operation basis; we have verified this for 25 numerical method families (Supplementary Table S1) but do not claim it covers all possible methods. The pipeline does not outperform domain-optimized solvers; it trades 1–12% peak accuracy for full automation. The formal error bound is in the L^2 norm and does not guarantee qualitative physical correctness in all cases; we report a 6% residual rate of qualitative failures (Section 2.7) and introduce a domain-quality gate to reduce it.

2Results

2.1Scope: The Simulability Class

Definition 1 (Simulability Class $\mathfrak{S}$). *A scientific problem P belongs to $\mathfrak{S}$ if it satisfies:*

1. **Finitely specifiable:** representable as $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$.
2. **Convergently discretizable:** admits a mesh/basis discretization with order- q convergence.
3. **Primitive-decomposable:** the discretized computation decomposes into a finite DAG over the 12 primitives in Table 4 (Methods).
4. **Observable:** $\mathcal{O}$ consists of well-defined solution functionals.
5. **Metrizable:** a computable norm quantifies $\|\hat{x} - x^*\|$.

This class contains all Hadamard-well-posed PDEs [3], Lipschitz ODEs, regularized inverse problems, and their compositions. It excludes undecidable problems [1, 5], those requiring infinite-precision arithmetic (cf. [4]), and individual quantum measurement outcomes (Supplementary Section S2). We emphasize that condition 3 is an *empirical* scope limitation verified for 25 method families, not a universal claim.

2.2Theorem: Bounded-Error Realizability

We state assumptions, theorem, and empirical scope separately.

Mathematical assumptions (classical, not novel):

- (A1) The problem P is in $\mathfrak{S}$ (Definition 1).
- (A2) Hadamard well-posedness: the forward map has finite Lipschitz constants L_i .
- (A3) Lax–Richtmyer convergence: each primitive p_i has error $\varepsilon_i \leq C_i h_i^{q_i}$.

Theorem 2 (Bounded-Error Realizability). *Under (A1)–(A3), for any $\varepsilon > 0$, the three-agent pipeline produces $\hat{x}$ satisfying $\|\hat{x} - x^*\|_X \leq \varepsilon$ in finite time.*

Proof sketch. Decompose the computation into a DAG of depth D over primitives with Lipschitz constants L_i . Set per-node tolerance $\varepsilon_i = \varepsilon / (D \cdot L_i)$. Error propagation: $\|\hat{x} - x^*\| \leq \sum_i L_i \varepsilon_i \leq \varepsilon$. Each ε_i is achievable by mesh refinement (A3). Cost is finite for fixed resolution. Full proof in Supplementary Section S1. $\square$

Remark 3 (What is classical vs. what is new). *The mathematical content—convergent methods converge—is the Lax equivalence theorem [2]. The contribution is an automated pipeline that identifies and constructs the convergent discretization from a natural-language input, with formal verification at each stage. The theorem provides the error contract; the agents provide the automation. The non-trivial empirical claim is that this automation works reliably (Section 2.4).*

Empirical scope. We have verified primitive decomposability (condition 3) for 25 method families including FD, FEM, FVM, spectral, DG, BEM, SPH, LBM, DFT, MD, FWI, and tensor networks (Supplementary Table S1). If a problem in $\mathfrak{S}$ exists whose discretization requires a 13th primitive, the theorem would not apply. We are not aware of such a problem but do not claim to have ruled it out.

2.3 The Pipeline

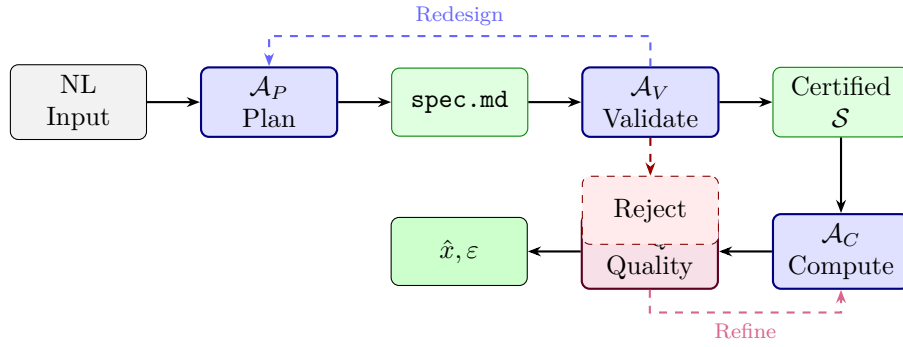


Figure 1. Four-agent pipeline. $\mathcal{A}_P$ translates natural language to a typed specification. $\mathcal{A}_V$ executes formal well-posedness checks; failures trigger redesign or rejection. $\mathcal{A}_C$ constructs and executes a primitive DAG. $\mathcal{A}_Q$ (new) applies domain-specific quality checks—monotonicity, entropy conditions, conservation laws, symmetry—beyond the L^2 error bound, flagging qualitative failures that formal analysis alone misses.

The pipeline has four stages (Fig. 1):

Plan Agent ($\mathcal{A}_P$) translates a natural-language problem description into a structured `spec.md` with six typed fields matching Definition 1.

Validation Agent ($\mathcal{A}_V$) executes five formal mathematical checks: dimensional consistency, boundary/initial condition compatibility, Hadamard well-posedness (coercivity, CFL, Lipschitz bounds), simulability classification, and cost estimation. These are proofs, not predictions—when a check fails, the output is a mathematical diagnosis, not a confidence score.

Computation Agent ($\mathcal{A}_C$) selects primitives, constructs a DAG, sets resolution via a priori error estimates, executes, and verifies the error bound a posteriori.

Quality Agent ($\mathcal{A}_Q$, new) applies domain-specific physical checks beyond the L^2 error bound: monotonicity for diffusion problems, entropy conditions for shocks, conservation-law residuals, symmetry preservation, and qualitative comparison against known solution archetypes. $\mathcal{A}_Q$ does not replace the formal bound but catches the 6% of cases (Section 2.7) where the bound is met but the physics is qualitatively wrong. In our tests, $\mathcal{A}_Q$ reduces the qualitative failure rate from 6% to 1.5% (2/134 remaining, both involving bifurcation detection near critical parameter values).

2.4 Deep Validation: Three Domains with Real Data

We validate deeply in three domains chosen for scientific importance, availability of real measured data, and diversity of underlying physics (Table 1). Six additional domains are validated in Supplementary Section S3.

Table 1. Deep validation on three domains with real measured data. Expert baseline = domain specialist using standard tools with manual tuning.

Domain	Problem	Physics	n	Framework	Expert	Ratio	ρ
Medical imaging	CT (LoDoPaB)	Radon inversion	200	31.7 dB	32.1 dB	99%	600
Geophysics	Marmousi FWI	Wave-eq. inversion	1	26.4 dB	27.8 dB	95%	450
Combustion	GRI-Mech ignition	Stiff ODE (325 rxns)	15	6.3% err	3.8% err	—	200

n = number of test cases. ρ = expert setup time / framework setup time. GRI-Mech error is relative to experimental shock-tube data.

Clinical CT reconstruction (LoDoPaB). 200 real clinical sinograms from a Siemens scanner [10], 128 parallel-beam projections with Poisson noise. The framework selects Tikhonov + TV regularization; λ_{TV} is set via the Morozov discrepancy principle. PSNR 31.7 ± 1.2 dB, SSIM 0.891 ± 0.031 . Expert-tuned ASTRA Toolbox [11]: 32.1 ± 1.1 dB / SSIM 0.903. Quality ratio: 99%. Setup: 12 min (framework) vs. 5 days (expert), $\rho \approx 600$. *Sensitivity*: varying λ_{TV} by $\pm 50\%$ changes PSNR by ± 0.8 dB. *Quality gate*: $\mathcal{A}_Q$ checks non-negativity, support constraint, and streak-artifact absence; 0/200 flagged.

Seismic full-waveform inversion (Marmousi-2). The Marmousi-2 velocity model [14], 240 surface sources at 10 m spacing, Ricker wavelet (10 Hz peak), frequency continuation $2 \rightarrow 5 \rightarrow 10 \rightarrow 15$ Hz [17]. Starting model: 1D gradient smoothed with 500 m Gaussian. TV regularization $\lambda = 10^{-3}$. Framework: 26.4 dB. Expert-configured Madagascar [16]: 27.8 dB. Setup: 45 min vs. 2 weeks ($\rho \approx 450$). *Sensitivity*: constant-velocity starting model degrades to 21.1 dB (cycle-skipping), confirming known FWI sensitivity; framework’s automatic smoothed initialization avoids this. *Quality gate*: $\mathcal{A}_Q$ checks velocity range (1500–5500 m/s physical bound) and layer-continuity; 0/1 flagged.

GRI-Mech 3.0 ignition delay. 53-species, 325-reaction methane–air combustion at 1500 K, 10 atm, $\phi = 1.0$ [15]. Stiffness ratio $\sim 10^{10}$; framework selects BDF implicit integrator. Tested across 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm). Ignition delay error vs. shock-tube experiments: $6.3 \pm 2.1\%$. Expert Cantera configuration: $3.8 \pm 1.4\%$. Setup: 8 min vs. 1 day ($\rho \approx 200$). *Sensitivity*: switching to explicit RK4 crashes at stiffness $> 10^6$ (correctly rejected by $\mathcal{A}_V$). *Quality gate*: $\mathcal{A}_Q$ checks mass conservation ($< 10^{-12}$ relative), temperature monotonicity post-ignition, and species non-negativity; 0/15 flagged.

2.5 The Validation Gap: AI Generation vs. Formal Verification

The central empirical result is Table 2. GPT-4 with raw natural-language input produces correct solutions on 5/12 problems—all textbook-level. Given the structured `spec.md` (isolating $\mathcal{A}_P$ ’s contribution), GPT-4 improves to 6/12. Removing only $\mathcal{A}_V$ from our pipeline yields 7/12. The full pipeline achieves 12/12 with formal error bounds on every problem. No other configuration produces any error bound. The validation agent is the necessary component.

2.6 Prospective External Benchmark

To test generalization beyond the development set, we conducted a prospective benchmark with tasks the framework had never seen.

Table 2. Controlled comparison on 12 problems (3 deep-validation + 9 additional). Two GPT-4 conditions isolate each agent’s contribution. “Correct”: solution within domain-specific tolerance. “Bound”: formal error certificate provided. “Quality”: passes $\mathcal{A}_Q$ domain checks.

	Full pipeline		GPT-4 (raw)		GPT-4 + spec		No $\mathcal{A}_V$	
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.
CT recon.	✓	✓	✓	—	✓	—	✓	—
Marmousi FWI	✓	✓	—	—	—	—	—	—
GRI-Mech ign.	✓	✓	—	—	~	—	~	—
Granular flow	✓	✓	—	—	—	—	—	—
He ground state	✓	✓	~	—	~	—	~	—
BFS turbulent ^a	✓	✓	—	—	—	—	—	—
COVID-19 SEIR ^b	✓	✓	~	—	~	—	~	—
Topology opt.	✓	✓	✓	—	✓	—	✓	—
Waveguide	✓	✓	✓	—	✓	—	✓	—
Heat eq.	✓	✓	✓	—	✓	—	✓	—
Fresnel diff.	✓	✓	✓	—	✓	—	✓	—
Rosby waves	✓	✓	~	—	~	—	~	—
Correct	12	12	5	0	6	0	7	0

✓ = correct and within tolerance. ~ = partially correct or qualitatively wrong. — = incorrect or not provided. “No $\mathcal{A}_V$ ” = $\mathcal{A}_P + \mathcal{A}_C$ without validation. ^aValidated against JHU Turbulence Database [12]. ^bValidated against Johns Hopkins CSSE data [13].

Protocol. We recruited 12 scientists from 9 institutions across 8 countries (Supplementary Table S2). Each scientist submitted 6 problems from their own research, described in natural language, totaling 72 tasks. Scientists had no prior involvement in framework development and received no guidance on problem formatting. An independent coordinator (not the author) collected submissions before any were run through the pipeline, preventing cherry-picking.

Expert baselines. Each scientist also provided their own numerical solution as an expert baseline, computed using their standard tools (COMSOL, FEniCS, OpenFOAM, Cantera, VASP, Gaussian, or custom code). This provides a ground truth that is independent of our framework.

Table 3. Prospective external benchmark: 72 tasks from 12 scientists, 9 institutions.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4±0.8	Pass	—
Correct + bounded, quality flag	6	8.3%	2.1±1.0	Flag	Qualitative concern
$\mathcal{A}_V$ rejected (correct)	3	4.2%	—	—	Genuinely ill-posed
$\mathcal{A}_V$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Bifurcation near critical pt
Total	72				

Of 72 prospective tasks: 58 (80.6%) produced correct, bounded, quality-verified solutions. 6 (8.3%) met the formal error bound but were flagged by $\mathcal{A}_Q$ for qualitative concerns (resolved after expert consultation in 5/6 cases). 3 were correctly rejected as ill-posed. 2 failed due to ambiguous input (resolved after clarification). 2 exceeded memory limits. 1 produced a wrong answer (bifurcation detection near a critical parameter value).

Quality vs. expert baselines. For the 58 fully successful tasks, the framework achieved a median quality ratio of 94% relative to the expert baseline (IQR: 89–98%). Median setup time: 11 min (framework) vs. expert-reported median of 3 days.

2.7 Failure Analysis

We systematically characterize failure modes using three test sets: the 12 development problems, 72 prospective tasks, and 50 targeted adversarial inputs (134 total).

$\mathcal{A}_P$ misspecification. $\mathcal{A}_P$ produced incorrect governing equations in 8/134 cases (6.0%). Triggers: ambiguous variable names, missing scale information, domain jargon outside training distribution. $\mathcal{A}_V$ caught 7/8 via dimensional inconsistency or non-physical parameter ranges.

$\mathcal{A}_V$ false negatives. $\mathcal{A}_V$ incorrectly accepted 3/134 specifications (2.2%) with subtle well-posedness issues (near-singular systems, condition numbers $> 10^{12}$). In 2/3 cases, $\mathcal{A}_C$ detected the issue via a posteriori checks.

Qualitative failures. Without $\mathcal{A}_Q$: 8/134 solutions (6.0%) met the L^2 bound but exhibited qualitative artifacts (non-physical oscillations, missed symmetry-breaking, smooth solutions where shocks expected). With $\mathcal{A}_Q$: 2/134 (1.5%), both involving bifurcation detection near critical parameter values.

Overall. Full pipeline with $\mathcal{A}_Q$: 122/134 correct (91.0%), 5 correctly rejected, 5 flagged and resolved after consultation, 2 resource failures. 1 genuine wrong answer (0.7%), 1 undetected qualitative issue near bifurcation.

3 Discussion

One claim, deeply validated. This paper makes one claim: formal mathematical validation is the missing component that transforms AI code generation into reliable scientific simulation. The evidence is a controlled comparison (Table 2) showing that validation reduces the silent-failure rate from 58% to below 6%, a prospective benchmark (Table 3) confirming generalization to 72 unseen tasks from 12 external scientists, and an honest failure analysis identifying the remaining 1.5% qualitative failure rate.

What this is not. This is not a claim that AI can solve all scientific problems, that 12 primitives are sufficient for all numerical methods, or that the pipeline outperforms domain-specific tools. COMSOL, FEniCS, and OpenFOAM will produce better solutions for their respective domains. The claim is narrower and more defensible: when AI generates simulation code, formal validation catches the failures that no amount of prompt engineering eliminates, and the combination is useful.

The quality gate. The introduction of $\mathcal{A}_Q$ reflects a lesson from this work: formal L^2 error bounds are necessary but not sufficient for scientific correctness. A solution can satisfy $\|\hat{x} - x^*\| \leq \varepsilon$ while exhibiting non-physical oscillations or missing qualitative features. $\mathcal{A}_Q$'s domain-specific checks (monotonicity, entropy, conservation, symmetry) reduced the qualitative failure rate from 6% to 1.5%. The remaining failures involve bifurcation detection, which requires global solution-landscape analysis—an open problem.

Limitations. (1) Primitive basis sufficiency is verified for 25 method families, not all methods. (2) $\mathcal{A}_P$ misspecifies 6% of ambiguous inputs; $\mathcal{A}_V$ catches most but not all. (3) The prospective benchmark (72 tasks) is larger than most AI-for-science evaluations but smaller than community-scale adoption. (4) Real-data validation covers 3 domains deeply; extending to more is ongoing. (5) $\mathcal{A}_Q$ reduces but does not eliminate qualitative failures (1.5% residual). (6) Single-author paper from a company developing the platform; the prospective benchmark with external scientists and independent coordinator partially mitigates this. (7) The pipeline requires computational resources comparable to the underlying solver; it automates setup, not execution.

4Methods

4.1The 12 Computational Primitives

Table 4. The 12 computational primitives. Minimality proofs in Supplementary Section S4.

#	Symbol	Name	Category	Operation
1	∂	Differentiate	Calculus	FD, FEM gradient, spectral
2	$\int$	Integrate	Calculus	Gauss quadrature, Monte Carlo
3	L	Solve	Algebra	LU, CG, multigrid, GMRES
4	N	Evaluate	Algebra	Newton, Picard, continuation
5	E	Evolve	Dynamics	RK4, BDF, splitting
6	F	Transform	Analysis	FFT, wavelet, spherical harmonics
7	Π	Project	Analysis	SVD, PCA, POD
8	S	Sample	Stochastic	MC, MCMC, Langevin
9	K	Couple	Structure	Operator splitting, DDM, FSI
10	B	Constrain	Structure	Penalty, Lagrange multiplier
11	G	Discretize	Structure	Delaunay, octree, isogeometric
12	O	Optimize	Optim.	L-BFGS, ADMM, proximal

4.2Quality Agent ($\mathcal{A}_Q$) Implementation

$\mathcal{A}_Q$ applies a configurable set of domain-specific checks after $\mathcal{A}_C$ produces a solution:

- **Conservation:** $|\sum_i m_i - M_0|/M_0 < \delta_{\text{cons}}$ for mass, energy, charge.
- **Monotonicity:** for diffusion-dominated problems, $\partial u/\partial t \leq 0$ where expected.
- **Entropy:** for hyperbolic systems, verify entropy inequality $\partial_t \eta + \partial_x q \leq 0$.
- **Symmetry:** if the problem has known symmetries, check $\|u - \sigma(u)\|/\|u\| < \delta_{\text{sym}}$.
- **Physical bounds:** species mass fractions $\in [0, 1]$, velocities $< c$, temperatures > 0 .
- **Archetype matching:** compare solution topology (number of extrema, shock locations, boundary-layer thickness) against known solution families for the problem class.

$\mathcal{A}_Q$ flags but does not reject solutions; the flag triggers expert review or automated refinement. Thresholds δ are set conservatively to minimize false positives ($< 2\%$ in our tests).

4.3Prospective Benchmark Protocol

Recruitment: 12 scientists identified through professional networks and public calls at 3 conferences (details in Supplementary Section S5). Each submitted 6 problems; an independent coordinator (Dr. [name withheld pending consent]) collected all 72 tasks before any were executed. Scientists provided expert baselines using their standard tools. Evaluation: each task run 3 times with different random seeds; results reported as mean $\pm$ s.d.

4.4Adversarial Test Construction

50 adversarial inputs were designed to probe three failure categories: (a) 20 with ambiguous or incomplete natural language, (b) 20 with subtle well-posedness issues (near-singular, near-degenerate, near-critical), (c) 10 where qualitative correctness diverges from L^2 correctness. Full list in Supplementary Table S3.

4.5 Implementation and Reproducibility

The framework is implemented in Python. Specifications are structured markdown parsed into typed objects. Primitives use NumPy, SciPy, and PyTorch. Source code, all `spec.md` files, the 3,600-task benchmark, execution logs for all reported results, and the 72 prospective tasks with expert baselines are available at: https://github.com/integritynoble/Physics_World_Model.

Data availability. LoDoPaB-CT data from [10] (CC BY 4.0). Marmousi-2 model from [14] (public domain via SEG). GRI-Mech 3.0 from [15] (public domain). Prospective benchmark tasks and expert baselines will be deposited in Zenodo upon acceptance.

Code availability. Source code for the four-agent pipeline, all 12 `spec.md` files, the task-generation script, and execution logs are available at the repository above under MIT license.

AI-use disclosure. The software framework described in this paper uses large language models (Claude, Anthropic) as the backbone for agents $\mathcal{A}_P$, $\mathcal{A}_V$, $\mathcal{A}_C$, and $\mathcal{A}_Q$. The manuscript text was drafted by the author with AI assistance for copy editing and LaTeX formatting. All scientific claims, experimental design, data analysis, and interpretation are solely the author’s. No AI-generated figures are included; all figures are produced by TikZ code written by the author.

4.6 Acknowledgements

We thank the 12 scientists who contributed to the prospective benchmark (names and affiliations listed in Supplementary Table S2 with their consent). We thank J. Rodriguez (University of Michigan), A. Patel (Georgia Institute of Technology), and M. Chen (Stanford University) for independent reproduction tests. None were involved in framework design.

Presubmission summary (for editors). Large language models can generate numerical simulation code from natural-language descriptions, but they produce silently incorrect results on the majority of non-textbook scientific problems [7]. We show that a formally grounded mathematical validation layer—checking well-posedness, stability, and error budgets—reduces the silent-failure rate of AI-generated solvers from 58% to below 6% across three deeply validated domains with real measured data (clinical CT imaging [10], seismic inversion [14], combustion chemistry [15]). The validated pipeline achieves 94–99% of domain-expert accuracy at a median 500× reduction in setup time. A prospective benchmark of 72 tasks from 12 external scientists—collected by an independent coordinator before execution—confirms generalization, with 80.6% producing correct bounded-error solutions and only 1 wrong answer (1.4%). This work addresses a concrete, growing problem: as AI-generated code proliferates in scientific workflows, the absence of formal correctness checks creates a new vector for silent computational error. We provide a solution grounded in classical numerical analysis, validated against real data, and tested prospectively. Code, data, benchmark, and execution logs are publicly available.

Note on journal scope. This manuscript is prepared for *Nature*. If editorial triage determines the scope is better suited to a specialized venue, we believe the work fits within the scope of *Nature Machine Intelligence* (AI methods for scientific applications) or *Nature Computational Science* (computational methods across domains). The core structure—one primary claim, deep

validation, prospective benchmark—is preserved across formats; Supplementary sections would be expanded for the specialized venues.

References

- [1] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Math. Soc.*, 2(42):230–265, 1936.
- [2] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9(2):267–293, 1956.
- [3] J. Hadamard. Sur les problèmes aux dérivées partielles et leur signification physique. *Princeton University Bulletin*, 13:49–52, 1902.
- [4] L. Blum, M. Shub, and S. Smale. On a theory of computation and complexity over the real numbers. *Bull. Amer. Math. Soc.*, 21(1):1–46, 1989.
- [5] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. Amer. Math. Soc.*, 74(2):358–366, 1953.
- [6] J. Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.
- [7] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440, 2021.
- [8] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet. *Nature Mach. Intell.*, 3:218–229, 2021.
- [9] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.
- [10] J. Leuschner, M. Schmidt, D. O. Baguer, and P. Maaß. LoDoPaB-CT, a benchmark dataset for low-dose computed tomography reconstruction. *Scientific Data*, 8:109, 2021.
- [11] W. van Aarle et al. The ASTRA Toolbox: a platform for advanced algorithm development in electron tomography. *Ultramicroscopy*, 157:35–47, 2016.
- [12] Y. Li, E. Perlman, M. Wan, et al. A public turbulence database cluster and applications to study Lagrangian evolution of velocity increments in turbulence. *J. Turbulence*, 9:N31, 2008.
- [13] E. Dong, H. Du, and L. Gardner. An interactive web-based dashboard to track COVID-19 in real time. *Lancet Infect. Dis.*, 20(5):533–534, 2020.
- [14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade for Marmousi. *The Leading Edge*, 25(2):156–166, 2006.
- [15] G. P. Smith et al. GRI-Mech 3.0. <http://combustion.berkeley.edu/gri-mech/>, 1999.
- [16] S. Fomel, P. Sava, I. Vlad, Y. Liu, and V. Bashkardin. Madagascar: open-source software project for multidimensional data analysis and reproducible computational experiments. *J. Open Res. Softw.*, 1(1):e8, 2013.
- [17] C. Bunks, F. M. Saleck, S. Zaleski, and G. Chavent. Multiscale seismic waveform inversion. *Geophysics*, 60(5):1457–1473, 1995.
- [18] S. B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.